

Problem Sets

Optimal Market Making for Cryptocurrency Options — The
Avellaneda–Stoikov Framework and its Extension to Options

North Carolina State University · Financial Mathematics

Summer 2026

NC STATE UNIVERSITY

Problem Sets

Optimal Market Making for Cryptocurrency Options

The Avellaneda–Stoikov Framework and its Extension to Options

North Carolina State University — Summer 2026

Instructions

These are the weekly programming assignments for Weeks 2–10 (Week 1 was the introduction week and had no graded set).

- **Problem sets are written in Python**, using **PyTorch** as the numerical engine. Use PyTorch autograd for all derivatives (Greeks, Jacobians, gradients) — you are **not** asked to hand-derive or hand-code algorithmic differentiation.
- **Week 7 is the exception**: it is written in **C++** and called from Python through a **pybind11** binding. A **CMakeLists.txt** scaffold is provided.
- Each problem set ships with a starter repository: data loaders, a test harness (**pytest**), and (Week 7) the **pybind11**/CMake scaffold.
- Emit numerical results as CSV and figures as PNG for inclusion in your write-up. Where a problem asks for a comment, print it from your code as a short summary of the actual numbers.

- Submissions are due Sunday 23:59 Eastern. Set seeds for reproducibility.

Worked solutions are distributed separately in the **Solutions** booklet after each deadline.

Problem Set 2 — Foundations of Market Microstructure (Python)

1. **Order-book reconstruction.** Implement a Python class `L2Book` that consumes a stream of events (`new`, `cancel`, `modify`, `trade`) and maintains the L2 state in a price-keyed structure (e.g. two `SortedDicts`). Apply it to one hour of Coincall BTC-PERP data and emit a CSV of mid-price and quoted spread over time. Remember to remove price levels that become empty after a cancel.
2. **Roll estimator.** Compute the Roll effective-spread estimator $2\sqrt{-\text{cov}(\text{Deltap}_t, \text{Deltap}_{t-1})}$ on the same sample and compare it to the directly observed bid-ask spread. Print both and their discrepancy.
3. **Glosten–Milgrom quote-setter.** Implement the rational quote-setter as a PyTorch belief update: given an informed-trade probability and a prior over the asset value, compute bid and ask from the Bayesian posterior after a buy or a sell. Unit-test against a hand-computed one-trade sequence.
4. **Glosten–Milgrom simulation.** Build a Monte Carlo simulator reusing problem 3 and emit a CSV of the public belief converging to the true value, averaged over many runs.
5. **Toxicity.** Compute realized spread and price impact over a 5-minute decomposition window on one hour of Coincall data; print a summary relating the numbers to order-flow toxicity.

Problem Set 3 — The Avellaneda–Stoikov Model (Python/PyTorch)

1. **Reduced ODE solver.** Implement a PyTorch backward integrator for the reduced system in `theta(t, q)` arising from the ansatz $V = -\exp(-\gamma(x+qS))\theta(t, q)$. Emit `theta` on a (t, q) grid.
2. **Closed-form quotes.** Implement the asymptotic reservation price $r = S - \gamma\sigma^2(T-t)$ and half-spread $\delta = (1/\gamma)\ln(1+\gamma/\kappa) + (\gamma\sigma^2/2)(T-t)$ as a reusable function, and `pytest` it against the solver from problem 1 where they should agree.

3. **Quote engine + simulation.** Simulate one trading day with `sigma = 2 USD/sqrts`, `kappa = 1.5`, `A = 140`. Emit the P&L distribution of the optimal policy alongside a symmetric, inventory-ignoring baseline.
4. **γ sensitivity.** Sweep `gamma` over three orders of magnitude and emit a CSV of inventory variance and Sharpe vs. `gamma`; identify in code the most effective regime.
5. **Price impact.** Extend the simulator so the mid reacts linearly to your own fills; run three impact levels and report how each degrades the optimal policy's P&L (independence-assumption violation).

Problem Set 4 — Stochastic Optimal Control (*Python/PyTorch*)

1. **Merton CRRA.** Solve the Merton problem under CRRA by discretizing the HJB (value iteration or backward ODE); confirm numerically the optimal portfolio *share* is constant in wealth.
2. **Merton CARA.** Repeat under CARA; confirm the optimal *dollar amount* is constant in wealth; emit a side-by-side CSV against problem 1.
3. **Almgren–Chriss.** Solve continuous-time optimal execution under quadratic transient impact; verify it matches the discrete-time result in the appropriate limit.
4. **HJB-residual verification.** Write a harness that evaluates the HJB residual for the Avellaneda–Stoikov value function (use autograd for the derivatives). Show the residual is $\sim$ machine zero for the true solution, then perturb it and show the residual diverges.
5. **Obstacle problem.** Solve the obstacle problem (projected SOR for the variational inequality) and compare to the unconstrained classical solution, demonstrating where a viscosity treatment is required.

Problem Set 5 — Avellaneda–Stoikov Engine (*Python/PyTorch*) — *Milestone 1*)

1. **Vectorized quote path.** Implement the quote computation vectorized over the inventory grid in PyTorch; benchmark it and report the latency distribution (mean, median, p95, p99).
2. **Intensity calibration.** Calibrate Λ^+ , Λ^- from one week of Coincall fill data by maximum likelihood (minimize the negative log-likelihood with `torch.optim`); report fitted `A`, `kappa`, and a χ^2 goodness-of-fit.
3. **Paper reproduction.** Reproduce the inventory/P&L behaviour of Avellaneda–Stoikov (2008) on simulated data; emit the figures as CSV; print a summary of any discrepancies.
4. **One-month backtest.** Build the event-driven harness (replay, fill simulator, P&L attribution) and run a one-month BTC-PERP backtest; report

mean/vol of P&L, Sharpe, max drawdown, mean and std of inventory, vs. a symmetric baseline.

5. **Tests and reproducibility.** Provide a `pytest` suite over the quote computation and cash/inventory bookkeeping; demonstrate reproducible backtests under a fixed seed.

Deliverable: Milestone 1 (see syllabus §2 for acceptance criteria).

Problem Set 6 — Volatility Surface and Calibration (*Python/PyTorch*)

1. **BS Greeks via autograd.** Implement Black–Scholes pricing in PyTorch and obtain delta, gamma, vega, theta, vanna, volga from autograd. Verify vanna/volga against finite differences; report the max relative error across moneyness/maturity.
2. **SVI + arbitrage checks.** Implement raw SVI and the Gatheral–Jacquier arbitrage-free constraint checks; apply to a deliberately broken parameterization and have the code report which constraints fail.
3. **SVI calibration.** Fit SVI to one Coincall BTC chain snapshot (`torch.optim`, autograd Jacobians); report parameters, per-strike residuals, and RMS error in vol units.
4. **SABR calibration.** Fit SABR to the same snapshot; emit a quantitative comparison against SVI and a printed note on strengths/weaknesses for crypto.
5. **Stability study.** Run daily SVI calibration over one month; emit the parameter time series; implement a smoothing scheme (seed each day from the previous fit) and report its effect.

Problem Set 7 — Fast Computation (*C++ with pybind11, called from Python — Milestone 2*)

A `CMakeLists.txt` and `pybind11` scaffold are provided. Build the module (`cmake -B build && cmake --build build`) and import it from Python as `fastmm`.

1. **COS / Carr–Madan pricer (C++).** Implement the Fang–Oosterlee COS pricer and the Carr–Madan FFT pricer for Heston European calls in C++ (use the provided FFT dependency). Bind both to Python. Verify against a PyTorch Monte Carlo benchmark and report COS convergence vs. N.
2. **Robust implied volatility (C++).** Implement an implied-vol inversion with a rational initial guess and a safeguarded Newton/Halley step; bind it. Show convergence across the surface, including deep wings and short expiries.

3. **Greeks, validated by autograd.** Expose prices from C++ and compute the Greek vector. Validate every Greek against a **PyTorch-autograd** reference implementation of the same pricer; report max relative error. (Do **not** hand-code adjoint AD in C++ — autograd is the reference.)
4. **Vectorized surface revaluation (C++).** Revalue the full Coincall BTC surface (300–500 contracts) with the Greek vector; release the GIL around the C++ loop; report per-option latency (mean, median, p95, p99) and the full-surface time (target < 5 ms).
5. **The binder, explained.** In a short `BINDING.md`, explain your `pybind11` module to a student new to it: how tensors cross the boundary without copying (buffer protocol / `py::array_t`), why and where you release the GIL, and how the CMake build finds Python and `pybind11`.

Deliverable: Milestone 2 (see syllabus §2 for acceptance criteria).

Problem Set 8 — Options Market Making and Hedging (Python/PyTorch)

1. **Multi-Greek inventory.** Implement inventory aggregation and the inventory dynamics under fills with a vector `q` in $\mathbb{R}^k$; unit-test the jump structure a fill induces.
2. **El Aoud–Abergel solver.** Solve the reduced multi-Greek problem under a quadratic penalty in PyTorch; verify in code it collapses to the Avellaneda–Stoikov quotes when one Greek and one option are tracked.
3. **Penalty-matrix calibration.** Calibrate the quadratic penalty matrix from the empirical covariance of delta/gamma/vega over one week of Coincall data; emit the matrix with each entry justified by the data.
4. **Gamma–theta–variance identity.** Simulate a continuously delta-hedged short call (autograd for the hedge ratio) and verify numerically the P&L equals `integral 0.5*Gamma*(realized - implied variance)`.
5. **Threshold hedger + engine.** Implement a delta-hedger triggering when `|Delta| > 0.1 BTC`, integrate it with the Week-5 engine on one week of data, and report hedging cost, risk reduction, and the quoting/hedging interaction.

Problem Set 9 — Backtesting, Risk, and Taker Flow (Python/PyTorch — Milestone 3)

1. **Event-driven backtest.** Implement the harness with deterministic replay; validate by running an identical config twice for bitwise results.
2. **Fill simulator.** Model fill probability as a function of quote distance and opposite-side depth; calibrate on a training month and validate on a disjoint holdout.

3. **Full backtest.** Run the multi-Greek engine on Coincall BTC options; report mean/std daily P&L, Sharpe (with block-bootstrap CI), max draw-down, and aggregated-Greek time series.
4. **P&L attribution.** Decompose daily P&L into spread, inventory, gamma, and hedging; emit a CSV for a stacked-area plot; assert the streams sum to total P&L.
5. **Drawdown forensic + taker analysis.** Identify the worst-drawdown day, report the environment, the positions that drove the loss, and a counterfactual. Using the guest lecture, characterize the taker flow on that day and how it affected your fills.

Deliverable: Milestone 3 (see syllabus §2 for acceptance criteria).

Problem Set 10 — Advanced Extension and Final Report (Python/PyTorch)

1. **Proposal.** Submit a one-page proposal (problem, method, expected results, risks) for your chosen advanced Avellaneda–Stoikov extension by end of day one.
2. **Implementation.** Implement the extension in PyTorch on top of the milestone infrastructure (multi-asset, adverse-selection, signal-driven, discrete-inventory, or robust).
3. **Evaluation.** Quantify the improvement (or not) over the Week-9 baseline using the same risk and P&L metrics, over the same window.
4. **Final report.** ≤ 30 pages, following the lecture’s structure, incorporating all milestone work.
5. **Final presentation.** Twenty minutes to the cohort and invited industry guests.